

Universidade Leiria • Oeste

Escola de Tecnologia e Gestão

Desenvolvimento de Aplicações Distribuídas

Distributed Applications

Definition, Characterization, Examples

Marco Monteiro and Ricardo Gomes

1. Definition
2. Characteristics
3. Examples of applications, architectures and components
4. Distributed Web Applications

Definition

Distributed Systems and Distributed Applications

Definitions

- A distributed system is a collection of independent computers that appears to its users as a single coherent system.

- A distributed application is a software application built on top of a distributed system, whose components run on different nodes but cooperate through communication.

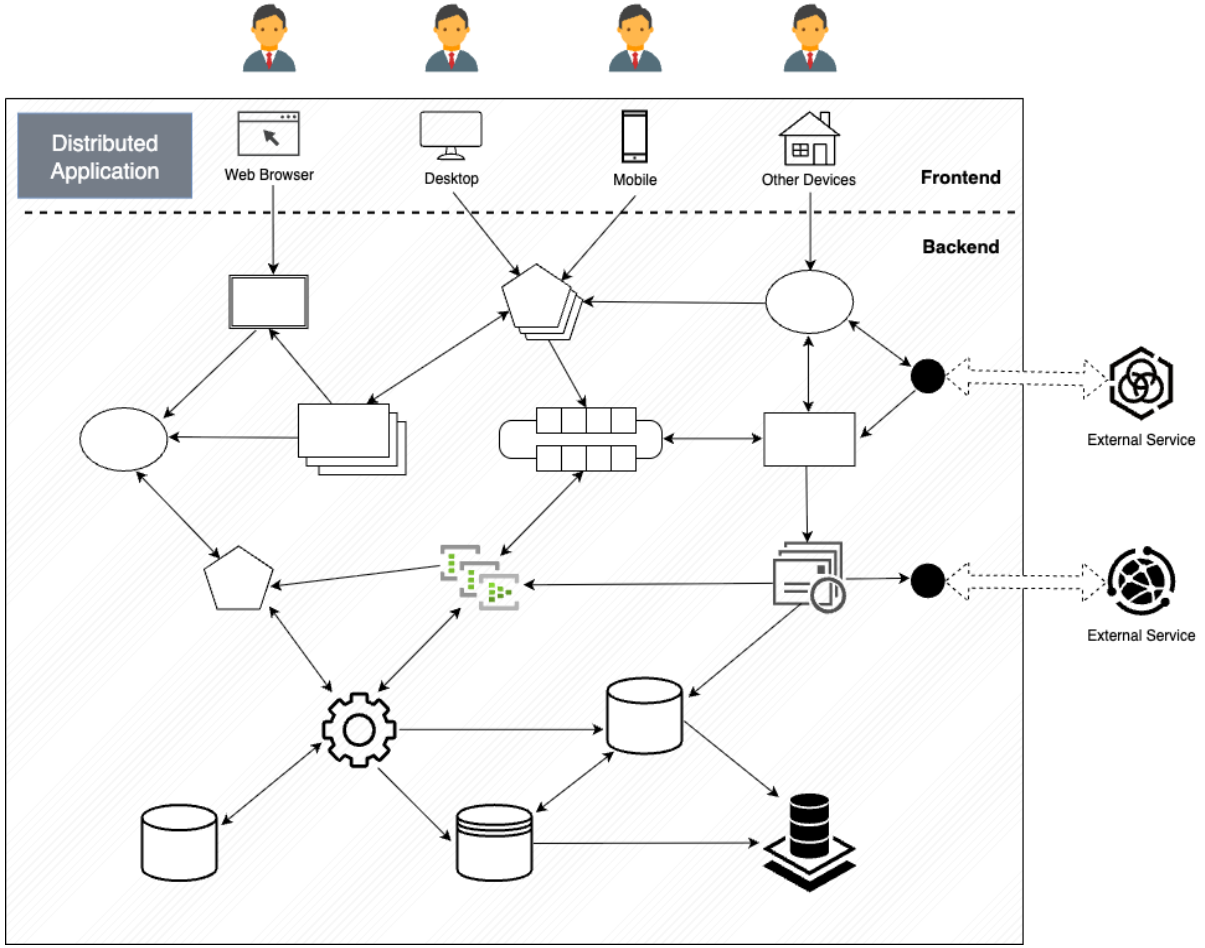
- Distributed System => Platform / infrastructure
  - Scalability, fault tolerance (reliability), transparency, resource sharing
- Distributed Application => Software built on top of the infrastructure
  - Leverages the capabilities of the underlying distributed system

Distributed Applications

- Example of a distributed application
  (Generic application with "non-specific" components)

- Multiple components (of different types) cooperate through communication
- End user perceives the system as a simple coherent system

  Internal complexity is hidden away from the end user, whose access and interaction are confined to the client-side (frontend) user interface components



Infrastructure

- In the context of distributed applications, nodes are logical execution environments where components of the application run and communicate. Nodes can vary in form and deployment. Examples of node types include:
  - Physical machines – Bare-metal servers that host application components directly on hardware.
  - Virtual machines – Software-defined machines providing an isolated OS environment on top of physical hardware.
  - Containers (e.g., Docker) – Lightweight, portable environments that package applications with their dependencies and share the host OS kernel.
  - Client devices – End-user devices that execute part of the application's interface, such as mobile phones, tablets, or desktop computers.
  - Web browsers – Execution environments for client-side application components (e.g., JavaScript or WebAssembly), typically running on client devices.

Characteristics

Main characteristics of Distributed Applications

Main Characteristics

- A distributed application is an application whose components run on multiple networked computers, communicating over the network, and aiming to provide transparency, scalability, concurrency, and fault tolerance, while also supporting heterogeneity across platforms, ensuring security in communication and access, and enabling efficient resource sharing.

- *These are desirable characteristics of distributed applications, but some may be absent in real-world cases*

Components Executing on Multiple Networked Computers

- Unlike monolithic applications, distributed applications are not confined to a single machine.

  In this context, “machine” should be interpreted broadly, encompassing not only physical computers but also virtual machines, containers and other isolated environments (e.g. Web Browsers)

- The application’s logic is distributed across multiple processes or services running on different nodes (clients, servers, databases, microservices, etc.).

  Each node effectively represents a "machine", regardless of whether it is physical, virtual, or containerized.

Network-Based Communication

- Components interact over a network using explicit communication (messages, RPC, REST, gRPC, message queues, etc.).

  Some distributed systems (e.g., on-chip distributed architectures, GPUs, or FPGA clusters) use shared memory or specialized buses instead of traditional network protocols

Transparency

- Application hides the complexity of distribution from users and developers:
  - Location transparency → users don't know where services/data reside.
  - Replication transparency → multiple copies appear as one.
  - Concurrency transparency → many users can access simultaneously without conflict.
  - Failure transparency → app keeps running despite node failures.

In practice, full transparency is rarely achieved — developers often deal with latency, partial failures, data consistency, and other issues arising from the distributed nature of the system.

Scalability

- The application can grow horizontally by adding nodes (servers, services, database shards).

  Not all distributed applications need to scale – scalability is desirable but not essential

  Cloud platforms commonly offer autoscaling, allowing applications to adjust resources based on demand

  Horizontal scaling of some components may require architectural adjustments. For example, web servers scale more easily when stateless

Note:

- Vertical scaling: increase resources (CPU, memory, storage) of a single machine*
- Horizontal scaling: add more machines* to distribute load

  ** "machines" can be physical, virtual, or containers*

Concurrency

- Supports many simultaneous clients and processes across nodes operating at the same time

  Requires synchronization and consistency management

  A distributed application could be used by one user only (e.g., a scientist running a distributed simulation on a cluster). Concurrency is common, but not strictly necessary.

Fault Tolerance / Reliability

- Designed to remain operational even if some nodes fail

  Achieved via replication, redundancy, consensus algorithms, and failover strategies

  - Replication → copying data across multiple nodes.
  - Redundancy → duplicate (cloning) nodes.
  - Consensus algorithms → methods for distributed nodes to agree on a common state in a distributed system.
  - Failover strategies → methods to automatically switch to a backup component or system when a primary one fails, ensuring continuity of service.

Heterogeneity

- The system may span different platforms (OS, hardware, cloud vendors) but must interoperate via standard protocols or middleware

Some distributed applications run in homogeneous environments (e.g., a Kubernetes cluster of identical Linux servers). Heterogeneity is common, but not essential.

Security

- Since components communicate across networks, security mechanisms such as authentication, authorization (access control), and encryption are essential

  Although security is fundamental in distributed applications, it is not an intrinsic property of distributed systems and is, unfortunately, often incorporated only at later stages.

Resource Sharing

- Components share resources across nodes: data, storage, services, and computation

Some distributed applications focus on coordination only without significant resource sharing.

Examples of applications, architectures and components

Examples of Distributed Applications

- Web-Based Distributed Applications
  Google Docs; Office 365; Facebook; Instagram; Gmail
- Mobile-Centric Distributed Applications
  WhatsApp; Uber; Spotify
- E-Commerce and Transactional Applications
  Amazon; eBay; AirBnb
- Enterprise Distributed Applications
  SAP ERP; Salesforce; Slack
- Streaming platforms
  Netflix; Amazon Prime; Spotify
- Massively Multiplayer Online Games (MMO)
  World of Warcraft; Fortnite
- Peer-to-Peer (P2P) Systems
  BitTorrent
- Blockchain Applications
  Ethereum, Hyperledger Fabric
- Many others . . .

Architectures

- Distributed applications have no single architecture, as they encompass several types of applications with fundamentally different structures and objectives.
- However, several well-known software architecture patterns have been adapted to the distributed context:
  - Client–Server – classic model where clients request services from centralized or replicated servers.
  - N-Tier – separates presentation, business logic, and data storage across distributed nodes.
  - SOA / Microservices – independent, networked services that cooperate
  - Event-Driven / Message Oriented – components communicate asynchronously via message brokers.
- Real-world distributed applications often combine multiple architectural patterns. There's rarely a "perfect" architecture; trade-offs exist between consistency, latency, cost, and complexity

Examples of components (1/3)

- Distributed applications include multiple components with different responsibilities. Examples:

- Client / Front-End:
  - Web App; Mobile App; Desktop App
- Compute / Backend:
  - Web Server – e.g., NGINX, serves HTTP requests and static content
  - REST API Server – e.g., Laravel, Express.js, exposes business logic via REST endpoints
  - gRPC Server – e.g., Go gRPC, handles high-performance RPC services
  - Serverless Function (FaaS) – e.g., AWS Lambda; event-triggered compute units
  - Background / Batch Worker – e.g., Celery; processes tasks asynchronously

Examples of components (2/3)

- Communication:
  - WebSocket Server – e.g., Socket.IO, persistent, full-duplex communication
  - Message Broker / Queue – e.g., RabbitMQ, handles reliable async message delivery
  - Async Messaging – e.g., Kafka, streams high-throughput events
- Storage / Database:
  - Relational DB – e.g., MySQL, structured data with tables and ACID transactions
  - NoSQL DB – e.g., MongoDB, unstructured data; scalability and flexible schemas
  - File / Object Storage – e.g., S3, large unstructured files (images, videos, etc.)
  - Cache – e.g., Redis, frequently accessed data in memory to reduce latency
  - Search / Index Engine – e.g., Elasticsearch, indexes and searches large volumes of data

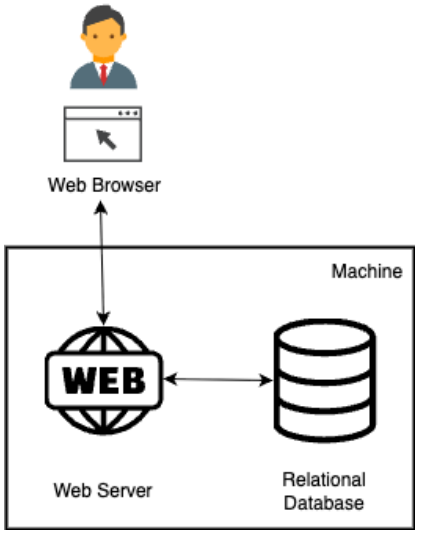
Examples of components (3/3)

- Infrastructure / Operations:
  - API Gateway – e.g., AWS API Gateway, routes requests and handles auth/rate limiting.
  - Load Balancer – e.g., NGINX, distributes traffic across servers.
  - Monitoring / Logging – e.g., Prometheus, collects metrics and logs.
- Security / Identity:
  - Authentication / Identity Service – e.g., Auth0, manages authentication and authorization.

Distributed Web Applications

When is a Web Application also a Distributed Application

Web Applications and Distributed Applications



- Are Web Applications also Distributed Applications?
  - Considering a Web Application with 1 web server and 1 relational database on a single machine
  - These are better described as client–server applications.
  - No definitive answer to the question

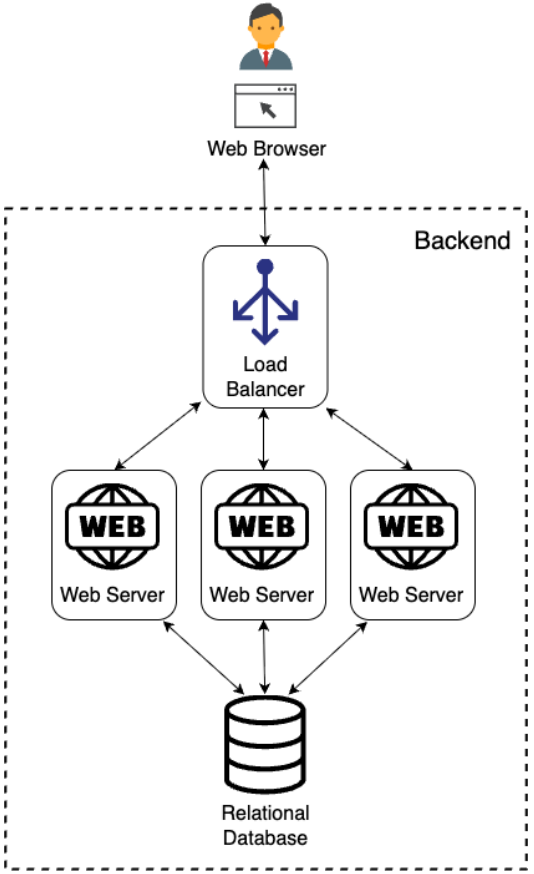
- Some authors consider client-server applications (as depicted) as the simplest form of distributed applications
  - This example includes 3 nodes (web browser; web server and database) that run on separate execution environments and communicate over a network
  - It captures the essence of distribution: separation of responsibilities and execution environments

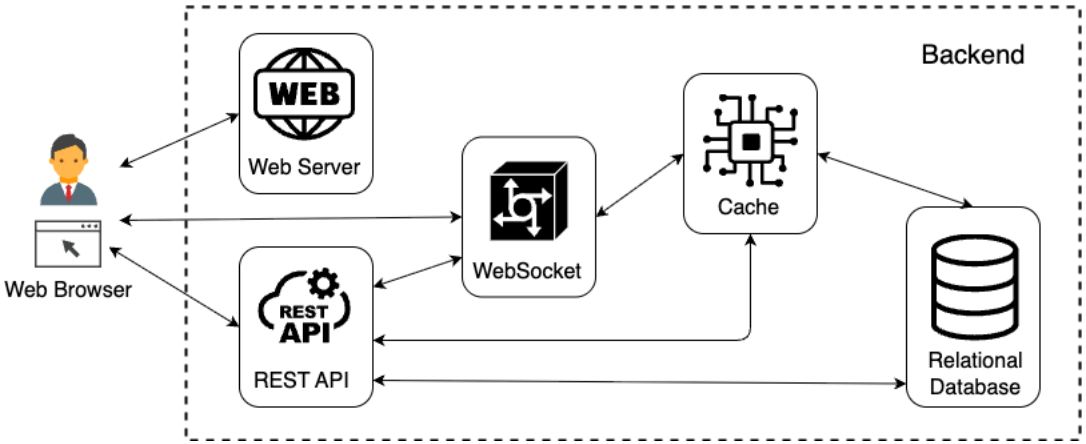
- Others, do not consider client-server applications as distributed applications, because they lack most of distributed applications characteristics
  - Distribution is minimal: single server and database limit independence, transparency and scalability
  - No complex coordination, replication, or fault tolerance
  - Communication is mostly synchronous and direct

Distributed Web Applications

- All authors agree that Web Applications are considered as Distributed Applications, when:

- All backend components have their own independent node (instead of everything running on a single machine), and:
  - Multiple web servers are behind a load balancer, or
  - Databases are distributed via replication or sharding across multiple nodes, or
  - New components are added - e.g., REST API Server, WebSocket Server, Message Broker / Queue, Cache, File / Object Storage, Authentication / Identity Service, etc.

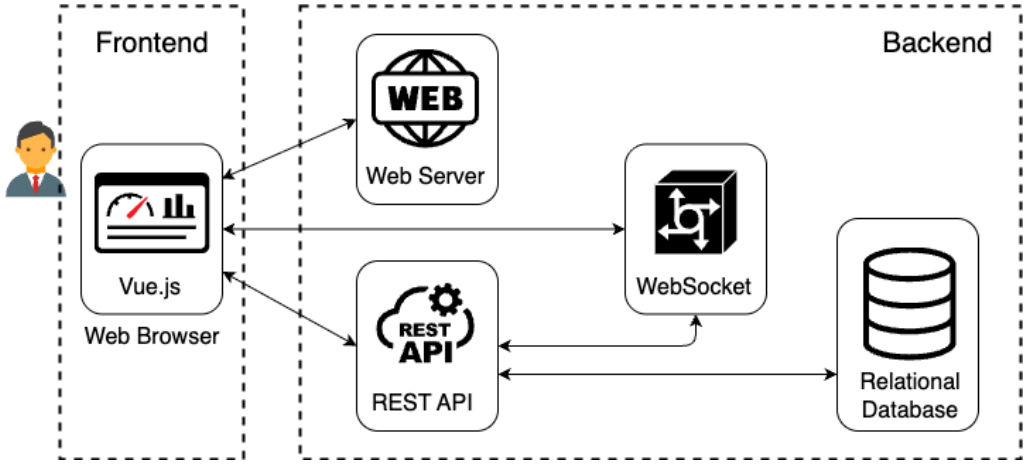




2 examples of Distributed Web Applications

Distributed Web Application used in this course

- The practical project for current course (DAD) will be a Distributed Web Application with the following minimum architecture:
- It is possible to add other components (e.g., queue server; cache; etc.)
- Minimum components:



- Web client application - SPA (Single Page Application) that runs in the browser. Implemented with Vue.js.
- Web Server – delivers the content to the browser (HTML; CSS; JavaScript – code for the web client application)
- REST API – server that exposes business logic through endpoints. Client application consumes this server to get data and execute operations
- WebSocket server – enables full-duplex communication for "real-time" messages between clients. It can also send or receive notifications to and from the REST API
- Relational database – structured data relative to the business

Bibliography

- Tanenbaum, Andrew S., & Van Steen, Maarten (2017). *Distributed Systems: Principles and Paradigms* (2nd ed.). Pearson.
- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.
- https://vfunction.com/blog/distributed-architecture/
- https://www.atlassian.com/microservices/microservices-architecture

Acknowledge

The authors acknowledge the use of ChatGPT (https://chatgpt.com/) and Google Gemini (https://gemini.google.com) as a tool supporting content development and text composition. Full responsibility for the final content rests with the authors.